

VolSeg Documentation; Settings Guide

The following guide gives a comprehensive overview of the possible settings, parameters and available function variables offered within the Volume-Segmantics Toolkit. This section should be used in conjunction with the [Functionality Documentation](#), where modes can be switched using the variables listed in the Training and Prediction .YAML files. This document is split into 3 parts for easier navigation including;

- Default Parameter Walkthrough.
- Training and Prediction Settings File Breakdown and Fine-Tuning Guide.
- Model Archetecture, Encoder and Loss Function list and Descriptions.

Default Parameter Walkthrough

After downlading the VolSeg Toolkit, the training_settings .YAML and prediciton_setting.YAML will be set to their default variable inputs. If significant changes are made to them during the course of your work and you require a fresh start, default version of these settings can be copied from [here](#).

For first time users, it is reccomended to famillierise yourselves with the most important inputs of each settings file as outlined in this sub-section, and how changing them in your initial tests might affect your model training and output success. From there, the fine-tuning section can then be used to further change inputs according to your user-case.

Training Settings YAML

These setting directly influence the model training command of the VolSeg Toolkit. These are located in the `2d_model_train_settings.yaml` file, where the individual inputs have comments as to their use next to their specific components; these are placed as reminders for their use and options. Further clarification for some of the more prominent and important settings and their possible changes are outlined below alongside their basic use (importance order);

- **model:type**; The designated Model Archetecture (overall training pathway) for your training; a full list of the potential archetctures, their uses and specific individual details can be found listed further on in this documentation. Prominent models include "U_Net" (default and most well rounded; it is reccomended that you test using this archetecture first), "U_Net_Plus_plus", "FPN", "DeepLabV3", "DeepLabV3_Plus" etc. though some may be more suitable depending on your user case.
- **model:encoder_name**; The designated Encoder used within your archetecture (feature reading and extraction); a full list of the potential encoders can be found listed further on in this documentation. The most tested and stable encoders include "tu-convnextv2_base", "tu_convnext_large", "resnet34" and "resnet50", though there has also been great success with the "efficientnet-b3", "timm-resnest50d"* and DINO (e.g. "dinov2_vitb14") encoders etc.
- **image_size**; The size of the data patches fed into the training command. This metric deals with how your data is stored and controlled after it is sliced. Input variables must be a multiple of 32, where the default input of 512 works great for image-label pairs between 250-700 cubic-voxels with efficient training times and moderate compute levels. The number refers to the size of the data patch your training data is further split into as the model reads its relivent features and components ready for

learning. If your compute capability is lower, or your data volumes are smaller, consider reducing this number (384 or 448). If you have a higher compute capability or your dataset is much larger, consider increasing this number (576 or 640); do take into account that increasing this variable will increase the training time and also have an impact on your prediction time aswell.

- num_cyc_frozen/num_cyc_unfrozen**; The number of runs you designate your data to train on with frozen and unfrozen parameter learning; the combined number equals the length of your training session. **The number of unfrozen epochs should be 2/3 the size of the frozen epochs; the default is 8/5.** When frozen, the 'backbone' of the model is trained, creating a 'within-task pre-trained model' from specific parameters while others are frozen in place. When unfrozen, no parameters are halted and the pre-existing 'within-task pre-trained model' is then fine-tuned using all available information. For further information regarding this methodology, look for 'transfer-learning' subject matter. The higher the number of epochs, the more training time the model will incorporate to reach its peak validation and evaluation conditions; be careful of *unfinished learning* and *over-prediction impacts* where the model will either show signs of not being given enough training time to reach optimum outcomes or will have reached its potential early and start to over-train and augment your model towards unwanted outcomes e.g. When looking at the *model_loss-Plot.png* of a very successful model, the 2 plot-lines should be reduce towards a straight line at the x axis (training and validation loss moving towards a stable horizontal progression representing a completion of learning). If the graph is still inclined or does not reach a stable horizon, this might mean the model is under-trained producing a prediction that may prove to be incomplete or lack complete feature boundaries. If the graph starts become erratic, this might mean the model is over-training producing a less accurate prediction visible on prediction outputs as noise, hashing artifacts and over-segmented areas. It is not recommended to decrease the epoch number from the default as successful models require a benchmark level of training time to produce successful outcomes; increasing the number of epochs will require more compute, longer training times and prove useful for larger training datasets, however they must also be carefully inspected to ensure loss rates remain stable throughout.
- loss_criterion**; The loss calculator used to evaluate the training of noted features through classifications and periodic evaluation and inform/guide successful continued model progression; a full list of loss functions can be found further on in this documentation. Depending on your user case, this should be changed according to your wanted segmentation outcomes as per the size and quality of the image-label inputs and projected outcomes; "CombinedCEDiceLoss" is the most adaptable and stable for validation over wide range of feature cases however great success has also been seen using "BCELoss", "DiceLoss" and "GeneralizedDiceLoss" etc.
- ce_weight/dice_weight**; The weighting for CombinedCEDiceLoss Loss critrion evaluation; ratio of the *cross-entropy* component (ce), helping stabilise imbalance over the segmentable classes, and *dice* component (dice), calculating the overlap between the predicted outcomes and groundtruth. This ratio must add to 1.0 (100%) or the training will fail under this loss function. The variables should only be changed if the image-label pair inputs are particularly large (1000+ cubic voxels), or the wanted features are numerous or complex in nature. The default is 0.2/0.8 split, however can be moved towards more equal terms (0.5/0.5) to suit the user case.
- eval_metric**; Evaluation metric analysing errors in segmentation model; this should be kept as MeanIoU unless your user case requires specific amendments based on its complexity or size.

Prediction Settings YAML

These settings directly influence the model prediction command of the VolSeg Toolkit. These are located in the `2d_model_predict_settings.yaml` file, where the individual inputs have comments as to their use next to their specific components; these are placed as reminders for their use and options. Further clarification for some of the more prominent settings and their possible changes are outlined below

The 'mirrored prediction settings' (`cuda_device`, `downsample` etc.) normalisation, augmentation, and 2.5D settings should be the same as those used in the training YAML for the training model; if you are unsure whether to change a prediction setting, look at the inputs fine-tuning information in the next sub-section.

- **quality**; The degree of prediction-image analysis and training model application. The 3 choices are linked directly to how the trained model is applied onto your prediction image; "low" quality refers to prediction along a single axis (images in the (x,y) plane if Z is chosen (default)) and "medium" and "high" quality refers to prediction along 3 axes or in 12 directions (3 axes, 4 rotations) respectively, before being combined by maximum probability. Medium is the default and will produce great outcomes with all inputs, though this can be lowered (low quality) if the prediction image is very large, or increased if your compute allows; high quality prediction will require much more compute and some user cases may not see significant improvement compared to the default outcome.
- **output_probs**; **New Feature**: Still in beta phase. should be kept as false.
- **output_entropy**; **New Feature**: Still in beta phase. should be kept as false.
- **prediction_axis**; The axis chosen when predicting using the low quality input.

When adapting training or prediction settings, start by varying these first based on your segmentation outcomes.

Training and Prediction Settings File Breakdown and Fine-Tuning Guide

File Breakdown

The training and prediction `.yaml` files are organised for easy navigation while amending training parameters. The training file is further split into 2 sections; *Basic* and *Advanced*, with the following sub-sections;

Basic

- Image and Model output
- Normalisation
- Reproducibility
- Model Architecture
- Learning rate
- Finder
 - Differential
 - Scheduler
- Loss Function
 - Selection
 - Legacy inputs
 - Advanced Losses
- Evaluation Metric

- Misc

Advanced

- Augmentation
- 2.5D slicing
- Multi-task
- Semi-supervised (SS)
- Pseudo-labeling SS

The prediction file is comprised of either 'continuating' variables (ones that should be mirrored from the training settings per the model utilised in the prediction command) or prediction specific inputs, with the following sub-sections;

- Prediction Specifics
- Sliding Window
- Normalisation
- Augmentation
- 2.5D Prediction

Fine-Tuning Guides

The following guide outlines what each input does, the possibilities for changing that settings and the potential affects of each settings line; it aims to inform users of usage prospects relative to their individual work case and help format your training/prediction parameters specifically towards you data.

2d_model_train_settings.yaml

- Basic;

Image and Model Output; File allocation settings

- **data_im_dirname**: *Name of folder that sliced data 2D images will be output to .*
 - The data is sliced and saved in this directory. This is temporarily stored in the VolSeg working directory while training is executed, and then deleted as the training command has completed. This should not be changed
- **seg_im_out_dirname**: *Name of folder that sliced segmentation 2D images with be output to*
 - The segmentation is sliced and saved in this directory. This is temporarily stored in the VolSeg working directory while training is executed, and then deleted as the training command has completed. This should not be changed
- **model_output_fn**: *Suffix for the saved model filename*
 - The model outfile file (containing your trained model) will have this suffix in the name. this can be changed to suit your preferred output name for project clarity (must contain no spaces).
- **data_hdf5_path**: *The internal HDF5 path to the image data*
 - Internal paths for data allocation when training. This should not be changed
- **seg_hdf5_path**: *The internal HDF5 path to the label data*
 - Internal paths for segmentation allocation when training. This should not be changed

Normalisation Settings; Data normalisation and processing variables

- **clip_data**: *Clip and rescale the image data intensities before saving to disk*
 - This argument allows the data input to be preprocessed for intensity constraint, limiting the values of the 3D array representing the data into a more consistent and stable format before it is sliced. The default is set to true and allows essential preparation of the dataset prior to training. This can be turned off, however is not recommended; a higher degree of success is seen with this remaining true.
- **st_dev_factor**: *The number of standard deviations from the mean to clip data to.*
 - The standard deviation used within as the data is clipped. The default of 2.575 corresponds to the 99th percentile of normal distribution; 99% of the data is preserved maintaining overall integrity, while extreme outliers are removed providing a reliable and smoother dataset before slicing occurs. This variable can be changed to exclude less or more outliers depending on your user case; it is recommended that this factor does not change as a reduction in your percentile may produce larger room for error in your predicted image.
- **minmax_norm**: *Min-max normalisation/rescaling. Only needed if clip_data if False.*
 - When a standard deviation is not provided through the clip_data input, this refers to the normalisation utility that rescales and image data for a fixed range. This should be kept as false when not in use, though if set to true will factorise data to a range of [0, 1]; this can be set to alternative ranges if your user case demands it e.g. [-1, 1] etc.
- **use_imagenet_norm**: *If True, applies ImageNet mean/std normalisation to input images (wanted if Imagenet pretrained weights used)*
 - Use normalisation factors specific to the Imagenet encoder prior to slicing. This is true by default; there are currently no other options for this variable as imagenet is the only encoder_weight available currently.
- **normalization_debug_mode**: *If True, enables detailed normalisation debugging in training loop.*
 - **Currently Improving Feature**: Still in testing. should be kept as false.

Reproducability; Random Seed Input for deterministic training and testing

- **random_seed**: *Optional integer seed.*
 - Consistent-condition integer input; When set to a number/integer the training will run under deterministic parameters relative to a 'seed', meaning that the independent and randomly generated variables will be set in place including the train/validation split, shuffling and data-loading workers reproducible. This is an ideal feature for those testing specifically the architecture or encoder habits with their data with stable training conditions. If set to zero, the seed will be generated randomly. If your model output is the main focus of a run, this should be left blank, however if the architecture or encoder behaviour relative to other statistics (for testing purposes) is the main focus, choose an integer and keep this consistent to produce a seed-generated outcomes.

Model Training Inputs; Overview of training settings

- **training_axes**: *Specify axes/single axis to train on. Choose from [All, Z, X, Y].*
 - The axis specified for training perspective. This should be kept to 'All', meaning the model takes into account all 3 axis directions when training, however can be changed; specific direction may be needed for highlighting features or mitigate artefacts, however may also

change the way the model views other comparable images. This should be used in the intermediate or testing stages of model creation.

- **image_size**: *Size of images used for training (must be multiple of 32)*
 - This refers to the size of the data/segmentation cubes used in the training command and how it is then sliced and handled when training begins; this number must be less than or equal to the overall image/seg file voxel size (ideally between 1/2 - 2/3 of its original size) and can be reduced and increased to improve model data-handling depending on image size though larger numbers used in this variable will require a higher compute load. If the image is very complex, it is suggested that you decrease this number to allow for more training cubes to be used.
- **downsample**: *If True, data will be downsampled by 2*
 - The data is downsized if set to true. This should only really be used if working with very large datasets for testing; data should be kept whole for best models.
- **training_set_proportion**: *Proportion of images to use the training, rest are used for validation*
 - The ratio used for training verses validation (15% validation is default, 85% for training) measured as a decimal; 1.00 is 100%. Changing this ratio may affect the amount of data available to the training; higher validation may mean that complex scan training may produce more varied results though varying slightly depending on user case can produce better models depending on wanted outcome.
- **cuda_device**: *The graphics card to use (between 0 and 3 for a machine with 4 GPUs)*
 - The graphics card designation if your computing setup has more than 1 (max 4); 0 is for 1 and first GPU.
- **num_cyc_frozen**: *Number of training epochs on frozen model*
 - The number of frozen training cycles (Epochs), increase/decrease per preferred training periods wanted (the ratio should be 2/3 frozen, 1/3 unfrozen); Increasing this might improve a model with incomplete training epoch leveling, however this will also increase your training time and may require more computing power.
- **num_cyc_unfrozen**: *Number of training epochs on unfrozen model*
 - The number of frozen training cycles (Epochs), increase/decrease per preferred training periods wanted (the ratio should be 2/3 frozen, 1/3 unfrozen); Increasing this might improve a model with incomplete training epoch leveling, however this will also increase your training time and may require more computing power.
- **patience**: *Number of epochs to wait before early stopping if validation loss does not improve*
 - The number of epochs the training protocol will wait if the validation score does not improve past a specific point. This will decrease the chance for training to start degrading with continued training time, though should be altered in conjunction with the epoch numbers if the data is complex.

Model Architecture Settings; Workflow of data-handling/processing used within the model trainer.

- **model: type**: *Choice of segmentation model type; 11 possible. U_Net is default and most stable (use Multitask_Unet for multitask advanced usage only).*
 - The Model Archetecture you wish to use for your training run; this can be from the range listed in the settings comment and must be exactly as described or an error will be given. *Unet* is the most diverse and effective on a wide range of images, *Unet++* is another alternative that may prove more effective on more complex label boundaries. *Multitask Unet* should only be used if the multitask functionality is being used.

- **model: encoder_name:** *Choice of encoder name; 19 possible. tu-convnextv2_base is default and most stable.*
 - The Model Encoder you wish to use for your training run; this can be from the range listed in the settings comment and must be exactly as described or an error will be given. *tu-convnextv2_large* is good for larger images, *resnet* can give better generalisations and performance on low contrast images, *other encoders* can prove effective when testing to correct specific image features.
- **model: encoder_weights:** *Pre-trained weights assignment.*
 - The default pretrained weights based on libraries within model architectures; compatible with all encoders available and needed for initialization.
- **model: encoder_depth:** *Number of feature levels trained as part of the encoder.*
 - When training, the model architecture downsamples the data to extract features; this depth limits the number of levels for the downsampling where higher steps mean more spatial resolution, and lower steps means faster training. The default is 5 and is the best for efficiency however can be increased if more compute is available or exact model is wanted. DINO encoders need should use less levels or the training may become very complex.
- **Commented out: _in_channels:** *Data layers for the model input tensor.*
 - Assigns the number of feature maps to expect; this is automatically assigned as per the data detection features, however can be commented in and assigned manually; 1 for grayscale, 3 for RGB, 5 for multi-scale or multi-modal data and N for 2.5D

Learning Rate Settings; Settings for finding the learning rate: the speed at which the model learns and makes predictions, where it finds an optimal balance between speed and accuracy.

- **lr_find_epochs:** *Number of training epochs for learning rate search.*
 - Number of epochs used to automatically find the numerical learning rate through a *test training*; finding the steepest decrease of loss through a gradual increase of the learning rate ensuring the model is training efficiently. This should be kept to one as higher values make the search slower with little benefit, though can be increased if needed.
- **lr_reduce_factor:** *Divisor for start and end LR when finding LR on reloaded model.*
 - This controls the optimization of the learning rate behaviour during training; by *decreasing* this number for *larger encoders*, it allows the learning rate to increase. This should be used in conjunction with the scheduler and multiplier settings to achieve better results. Default is 500 (medium sized encoder size, decrease per smaller encoder size).
- **encoder_lr_multiplier:** *Encoder LR multiplier for unfrozen training.*
 - Encoders scaler for the learning rate search with respect to the decoder. A smaller number leads to slower updates to the encoder and more suited for larger encoders, and larger numbers are used for smaller encoders and faster updated to the encoder. If set to 1.0 or none, the learning rate will be the same for the encoder and decoder. This should be used in conjunction with the scheduler and reduce_factor settings to achieve better results. Default is 0.1 (larger encoder settings, increase per smaller encoder size).

Learning Rate Scheduler; Settings directly tuning learning rate as training starts.

- **pct_lr_inc:** *The percentage of overall iterations where the LR is increasing.*
 - The number of total training steps spent increasing the learning rate from minimum to maximum; controls how quickly the learning rate ramps up at the start of the training.

Small values (0.05 min.) mean its peak is reached quickly leading to potential model instability, and larger numbers (0.4 max.) can stabilise training of larger models and those which data is particularly noisy. Numbers outside this range are not recommended. Default is 0.3 (30% of steps used for warm up)

- **starting_lr**: *Lower bound of learning rate search.*
 - This is the initial learning rate value from which the scheduling or decay starts from; as the training starts this will adjust over time to suit the training conditions. A higher starting rate can speed up convergence but risks overshooting the minimum rate required, however a lower learning rate, though proving to be more stable does increase the training time substantially. $5e-5$ is the default though can be adjusted per the user case.
- **end_lr**: *Upper Bound of learning rate search.*
 - This is the final learning rate at which training will cease, where the increase from its the *starting_lr* is and its measured loss is monitored to find the optimal learning rate range. A final smaller ending rate should finish the training with a low learning rate, which is better for fine-tuning, however a larger ending rate means the learning rate will remain high at the end helping to avoid overfitting but may result in reduced model stability. $1e-3$ is the default though can be adjusted per the user case.

Loss Function Settings; Choice of criterion and relative variables

- **loss_criterion**: *Choose from one of the 9 loss functions listed.*
 - The Loss Function you wish to use for your training run; this can be from the range listed in the settings comment and must be exactly as described or an error will be given. *CombinedCEDiceLoss* is the default and works very well on single, multiclass data and for larger prediction images. Other functions may work better for more complex images, or with specific user case features.
- **ce_weight**: **Used specifically for CombinedCEDiceLoss**; *Weight for Cross-Entropy component (alpha in CombinedCEDiceLoss).*
 - Used in conjunction with *dice_weight* to form a ratio; control loss function control depending on user case. CE; pixel-wise classification accuracy: higher number better for lower classes or equal class weight in dataset, lower for multiclass or more complex images.
- **dice_weight**: **Used specifically for CombinedCEDiceLoss**; *Weight for Dice component (beta in CombinedCEDiceLoss).*
 - Used in conjunction with *ce_weight* to form a ratio; control loss function control depending on user case. Dice; class imbalance: higher number better for more complex and datasets with high class variability or number, lower for less lower multiclass complexities.
- **dice_weight_mode**: *Dice weighting mode for class-weighted losses.*
 - The weight normalisation for smaller classes in multi-class training; *inverse_sqrt_freq* is the default and the best for more complex and multiclass data increasing the importance of the smaller classes with respect to the larger ones, *inverse_freq* prioritises the smaller classes over the larger ones, and *uniform* keeps the smaller and larger classes in an unbalanced dataset equal in terms of importance.
- **exclude_background_from_dice**: *Flag for the inclusion of the background class in metrics.*
 - This should be set to *True* when the background forms a large portion of the image compared to your wanted labels (roughly 80% background or more depending on image

data and complexity) and helps to focus the model on the segmentable features; helps to predict accurately and reduce noise.

- **alpha:** **Used specifically for BCEDiceLoss:** *When BCEDiceLoss selected, weighting for BCELoss.*
 - Preferred metric before being depreciated for CombinedCEDiceLoss. Depreciated argument.
- **beta:** **Used specifically for BCEDiceLoss:** *When BCEDiceLoss selected, weighting for DiceLoss.*
 - Preferred metric before being depreciated for CombinedCEDiceLoss. Depreciated argument.

Evaluation Metric; Performance and Validation Assesment

- **eval_metric:** *The Evaluation Metric used throughout training for model accuracy and validation.*
 - *Mean Intersection over Union (MeanIoU);* the mean value of the calculated IoU for each class, taking into consideration the true posatves fale posatives and false negatives from the predcition relative to the ground truth. It gives a stable and accurate value from which to interpret overlap and take into consideration class imbalance; prefect segmentation is 1.0 (100%). This is the default and the most widely used metric.
 - *DiceCoefficient;* Calculates a dicescore relative to the overlap between groundtruth and prediction, and better imbalanced class distributions; however cannot be used in conjunction with loss functions that include DiceLoss (CombinedCEDiceLoss, DiceLoss, GeneralizedDiceLoss and ClassWeightedDiceLoss) as it introduced biases.
- **dice_averaging:** *Which averaging system to use when dicecoefficient is used as the avaluation metric.*
 - **Used specifically for DiceCoefficient.** a chooce of 2 averaging methods while utilising dicecoefficient evaluation; "macro" averages the dicescore per class to take in overall model performance, whereas "weighted" takes into account each class individually to analyse each classes performance.

Misc. Training Settings; Additional training settings and input arguements.

- **plot_lr_graph:** *Output a graph of learning rate progression alongside other metrics.*
 - **Currently Improving Feature:** Still in testing. should be kept as false.
- **use_sam:** *Use Segment Anything Model (SAM-UNet) Resources.*
 - **Currently Improving Feature:** Still in testing. should be kept as false.
- **adaptive_sam:** *Use Adaptive Segment Anything Model (SAMA-UNet) Resources.*
 - **Currently Improving Feature:** Still in testing. should be kept as true (adaptive sam is defult for using SAM inputs).
- **encoder_weights_path:** *Input for alternative encoder usage.*
 - This points to the path of a potential encoder not yet listed in the featured toolkit. Used to point to data source files for testing new encoder files and edits; developmental usage.
- **full_weights_path:** *Input for alternative archetecture weight inputs.*
 - This points to the path of other training weights in the featured toolkit. Used to point to data source files for initial testing; depreciated usage.

- Advanced;

Augmentation Settings; Image-Label Data Training Library

- **augmentation_library:** *Choose augmentation library.*

- Assign the augmentation library to your training depending on user case; this is further explained in the [functionality documentation](#).
- **use_monai_datasets**: *Flag for specific MONAI datasets.*
 - When MONAI is selected as the augmentation_library, you can choose to use its datasets alongside others intergeted into the augmentation libruaries; this can sometimes improve the segmentation of medical images. This should be set to tru when MONAI augmentation libruary is selected (will have no bearing on albuementations if selected and remains true).

2.5D Slicing Settings; 2.5D functionality.

- **use_2_5d_slicing**: *Enables 2.5D functionality.*
 - Set to false when not using; must mirror prediction settings if model using 2.5D settings is created. Set to true when in use; will use num_slices and slice_file_format as further flags.
- **num_slices**: *Number of slices to use.*
 - Choice of the number of slices that shoudl be separated before selecting a central training slice; must be an odd number (3/5/7/9 etc.). This is further explained in the [functionality documentation](#).
- **slice_file_format**: *File format for multi-channel slices.*
 - The file format used when processing slices before training starts. Use .png for 3 slices mode (3 channels) or .tiff for 5 slices or above (over 3 channels).
- **skip_border_slices**: *Flag to skip start and end slice from slice selection.*
 - If enabled, will ski the first and last slice when choosing enacting the num_slices choice. should be kept to false as this function is not yet fully implimented; still being tested

Multi-task Learning settings; Multi-Task Functionality; boundary maps

- **use_multitask**: *Enables Multitask functionality.*
 - Set to false when not using and set to true when in use; will use subsequent function flags.
- **num_tasks**: *Number of tasks.*
 - Number of tasks integrated into the multitask designator; includes 0 as image and 1 as label files, and task 2 as the boundary file. This is further explained in the [functionality documentation](#).
- **decoder_sharing**: TBC
 - **New Feature**: Still in beta phase. should be kept as "shared".
- **seg_loss_weight**: TBC
 - **New Feature**: Still in beta phase. should be kept as 1.0.
- **boundary_loss_weight**: TBC
 - **New Feature**: Still in beta phase. should be kept as 1.0.
- **task3_loss_weight**: TBC
 - **New Feature**: Still in beta phase. should be kept as 1.0.
- **boundary_loss_type**: TBC
 - **New Feature**: Still in beta phase. should be kept as "bce".

Semi-Supervised Learning Settings; Self-Supervised Learning Functionality; Mean-Teacher and Pseudo-labeling

- **use_semi_supervised**: *Enables Self-supervised functionality.*
 - Set to false when not using and set to true when in use; will use subsequent function flags.

- **unlabeled_batch_size**: TBC
 - **New Feature**: Still in beta phase. should be kept as 8.
- **consistency_weight**: TBC
 - **New Feature**: Still in beta phase. should be kept as 0.1.
- **rampup_start**: TBC
 - **New Feature**: Still in beta phase. should be kept as 0.
- **rampup_end**: TBC
 - **New Feature**: Still in beta phase. should be kept as 10000.
- **ema_decay**: TBC
 - **New Feature**: Still in beta phase. should be kept as 0.99.
- **mean_teacher_vis_epoch_interval**: TBC
 - **New Feature**: Still in beta phase. should be kept as 5.
- **use_pseudo_labeling**: TBC
 - **New Feature**: Still in beta phase. should be kept as false.
- **pseudo_label_confidence_threshold**: TBC
 - **New Feature**: Still in beta phase. should be kept as 0.95.
- **pseudo_label_confidence_method**: TBC
 - **New Feature**: Still in beta phase. should be kept as "max_prob".
- **pseudo_label_min_pixels_per_class**: TBC
 - **New Feature**: Still in beta phase. should be kept as 10.
- **pseudo_label_use_teacher**: TBC
 - **New Feature**: Still in beta phase. should be kept as true.
- **pseudo_label_weight**: TBC
 - **New Feature**: Still in beta phase. should be kept as 1.0.
- **pseudo_label_rampup_start**: TBC
 - **New Feature**: Still in beta phase. should be kept as 0.
- **pseudo_label_rampup_end**: TBC
 - **New Feature**: Still in beta phase. should be kept as 5000.
- **pseudo_label_threshold_schedule**: ?TBC
 - **New Feature**: Still in beta phase. should be kept as "fixed".
- **pseudo_label_start_threshold**: TBC
 - **New Feature**: Still in beta phase. should be kept as 0.9.
- **pseudo_label_target_acceptance_rate**: TBC
 - **New Feature**: Still in beta phase. should be kept as 0.3.
- **pseudo_labeling_vis_epoch_interval**: TBC
 - **New Feature**: Still in beta phase. should be kept as 5.

2d_model_predict_settings.yaml

Prediction Settings: Fundamental prediction settings

- **quality**: *Degree of Prediction calibre.*
 - Quality can be set to 3 setting depending on need and compute; Low: predicts along a single axis (Specific-axis in 4 directions) giving a 2D prespective prediction from your model along a specified axis, Medium: predicts along 3 axis (Z/Y/X-axis) giving a well rounded 3D perspective prediction from your model. High: Predicts along 3 axis in 4 directions per dimension (Z/Y/X-axis in 4 directions) giving a higher quality '12-way' 3D

perspective prediction from your model along all axis. Setting the quality to high will require a larger computing threshold and take 3X longer than a normal prediction.

- **output_probs**: *Probability map feature.
 - **New Feature**: Still in beta phase. should be kept as false.
- **output_entropy**: *Entropy map feature.*
 - **New Feature**: Still in beta phase. should be kept as false.
- **cuda_device**: *The graphics card to use (between 0 and 3 for a machine with 4 GPUs)*
 - The graphics card designation if your computing setup has more than 1 (max 4); 0 is for 1 and first GPU.
- **downsample**: *If True, data will be downsampled by 2*
 - The prediction image data is downsized if set to true. This should only really be used if working with very large datasets for testing; data should be kept whole for best predictions.
- **one_hot**: *Enable one-hot encoded data output.*
 - **New Feature**: Still in beta phase. should be kept as false.
- **prediction_axis**: **Used for Low Quality prediction settings.** *Specify axis for low quality predictions.*
 - Set the axis for the low quality prediction setting; can be Z, Y or X axis. Z-axis is recommended as data is first sliced in this direction.
- **output_size**: *Size of images used for prediction (must be multiple of 32)*
 - This refers to the size of the data cubes used in the prediction command; this number must be less than or equal to the overall image/seg file voxel size (ideally between 1/2 - 2/3 of its original size) and can be reduced and increased to improve model data-handling depending on image size though larger numbers used in this variable will require a higher compute load. This number can be different than that used in the model training settings used for the training command though must reflect the prediction image needs; the default is 512, though should be changes to suit.
- **data_hdf5_path**: *The internal HDF5 path to the image data*
 - Internal paths for data allocation when predicting. This should not be changed

Sliding Window Interface; Entropy Map Variables **Only used if output_entropy is true**

- **use_sliding_window**: TBC
 - **New Feature**: Still in beta phase. should be kept as false.
- **sw_roi_size**: TBC
 - **New Feature**: Still in beta phase. should be kept as [512, 512].
- **sw_overlap**: TBC
 - **New Feature**: Still in beta phase. should be kept as 0.25.
- **sw_batch_size**: TBC
 - **New Feature**: Still in beta phase. should be kept as 4.
- **sw_mode**: TBC
 - **New Feature**: Still in beta phase. should be kept as gaussian.

Mirror settings; Normalisation

- **clip_data**: *Clip and rescale the image data intensities before saving to disk*
 - Allows the data input to be preprocessed for intensity constraint; essential to prediction image normalisation compared to a normalised training image; must be the same as training settings input.

- **st_dev_factor**: *The number of standard deviations from the mean to clip data to.*
 - The standard deviation used within as the data is clipped; the data is preserved maintaining overall integrity, while extreme outliers are removed providing a reliable and smoother dataset before slicing occurs. Must be the same as training settings input.
- **minmax_norm**: *Min-max normalisation/rescaling. Only needed if clip_data if False.*
 - Normalisation utility that rescales and image data for a fixed range when not using clip_data. Must be the same as training settings input.
- **use_imagenet_norm**: *If True, applies ImageNet mean/std normalisation to input images*
 - Use normalisation factors specific to the Imagenet encoder prior to slicing. Must be the same as training settings input.

~ Check that the prediction normalisation settings match those used for training model; this is imperative for uniformity and model consistency.

Mirror settings; Augmentation

- **augmentation_library**: Choose augmentation library.
 - Assign the augmentation library to your prediction command depending on user case; this is further explained in the [functionality documentation](#), though should be the same library used for the training model used in the prediction command.

Mirror settings; 2.5D Prediction

- **use_2_5d_prediction**: Enables 2.5D functionality.
 - Set to false when not using; must mirror training settings if model using 2.5D settings during training is used in the prediction command. Set to true when in use; will use num_slices and prediction_padding_factor as further flags.
- **num_slices**: Number of slices to use.
 - Choice of the number of slices that should be separated before selecting a central prediction slice; must be an odd number (3/5/7/9 etc.). This is further explained in the [functionality documentation](#), though should be the same number used for the training model used in the prediction command.
- **prediction_padding_factor**: Multiplier for padding amount
 - **Currently Improving Feature**: Still in testing. should be kept as 1.0.

~ As newer versions of the toolkit are released, further settings may be added and as such this documentation guide will be updated.

Model Architectures, Encoder and Loss Function Lists;

The choices available within the toolkit have been chosen specifically for their use in biomedical image segmentation. The variety listed have been tested using a range of datasets for capturing both larger and smaller features from multiple scan types and image sources. For more information regarding these choices, links have been provided for their relevant literature descriptions and details.

There are 10 current model architectures compatible with VolSeg. These can be changed via the Training Setting YAML where available options are also listed in the comments of the *Model Architecture* input; **U-Net** is the default and the most widely-tested architecture. The full architecture list includes:

- **U-Net** (Default)

A U-shaped convolutional neural network incorporating a contracting and expanding path on both sides (encoder and decoder) and a 'bottleneck' in the middle; as data passes through the network at specific levels feature maps, are directly passed from the encoder to the decoder towards corresponding layers, creating a bending shape (U-shape) that captures a wide degree of context from the whole images alongside smaller details; it was specifically designed to work with small datasets and medical imaging. More information on this architecture can be found [here](#).

- **U-Net++**

A U-Net incorporating nested skip connections and additional classification and intermediate layers to improve feature extraction and segmentation accuracy. The additional connections help to bridge the gap between the feature maps across the U-shape, allowing for a more deeply supervised network that proved more effective on more complex or multi-class data samples. More information on this architecture can be found [here](#).

- **FPN**

A Feature Pyramid Network (FPN) incorporates stepped leveling system; a bottom-up pathway (encoder), that resolves a series of 'backbone' feature maps at a wider degree of spatical levels, and a top-down pathway (decoder), that 'zooms into' the more abstract features to help the detection of smaller components. Combining the larger and smaller learned information through its lateral connections at different levels creates rich and comprehensive view of features from all levels of data input within the network. This architecture is very good at larger image predictions containing smaller wanted features. More information on this architecture can be found [here](#).

- **DeepLabV3**

A linear network that primarily uses both Atrous Convolutions; a process where spaces are added between kernel elements to expand the receptive field of learning without losing spatical resolution, and Spatical Pyramid Pooling; a method that probes the data at multiple layers and FOVs to capture further image context. It uses a Resnet-101 DCNN model as the parallel backbone where the convolutions are arranged as a cascade, funneling into a final model output. This architecture is particularly good at large feature recognition. More information on this architecture can be found [here](#).

- **DeepLabV3+**

A modified network where the DeepLabV3 is treated as the encoder and a separate decoder containing low-level features is then concatenated together to form a refined model output. this format is very good at realising object boundaries amongst more complex 'background' features. More information on this architecture can be found [here](#).

- **MA-Net**

A Multi-scale Attention Network (MA-Net) incorporates a self-attention mechanism within a modified Res-block linear network that integrates local feature recognition alongside their wider contextual dependencies. It uses a position-wise attention block and multi-scale fusion attention block within its network that help to both model the spatical dimensions and channel dependencies producing a high-performance map that helps inform model learning. The network has proved superior to other 2D

CNNs, however future 3D versions are still being developed. More information on this architecture can be found [here](#).

- **LinkNet**

A cyclical network of 2 halves; a series of encoder blocks (iterative convolution phases and feature maps of specific sizes for high level features) and a series of decoder blocks (directly linked to its encoder counterpart which then unsamples back to the original image size). These blocks have skip connections that help reduce the loss of spatical information and help the decoder use less parameters, meaning it is more lightweight than other models without comprimising to much performance. More information on this archetecture can be found [here](#).

- **PAN**

A Pyramid Attention Network (PAN), that combines a Resnet encoder attemtion mechinism and global attention upsample decoder to create a cyclical network that helps to capture multiple comparable features of differing sizes more accuratly. The Resnet informs a feature pyramid attension module (FPAM) of different scale feature information, which then feeds back into a global average pooling to form a guide for later combination steps. More information on this archetecture can be found [here](#).

- **SegFormer**

A linear network that is comprised of transformer-based encoder batches and a multi-layer perceptron decoder. The transformers (MiTs) do not require positional encoding, leading to increased perfomance over varies testing resolutions, and the custom simple decoder system provides a larger receptive feild for training parameters and learning. It combines powerful training with efficient compute loads and is particularly useful on large-image/mask datasets. More information on this archetecture can be found [here](#).

- **Vanilla U-net**

Currently in testing TBC

There are also 19 pre-trained encoders that can be used alongside these architectures: their individual usage relative to the model structure chosen is detailed alongside each description (curently to be reviewed and updated). **tu-convnextv2_base** is the default encoder and the most widely tested. The full pretrained encoder list includes:

ConvNeXt/ConvNeXt_V2: a feature extraction backbone based on Vision Transformers

- **tu_convnext_base**

Developed by Facebook AI research (2022), it progressively extracts hierarchacal features from images using a number of key component layers; a Stem layer (similar to ViT patch embedding), intermediate downsampling layers and feature resoluton stages. It is noted to perform very well on medical images and highly efficient and scalable when combined into UNet-like model structures. The base version has ~88M trainable parameters, [128, 256, 512, 1024] conv. channel dimension blocks and is built for moderate GPU memory capabilities and general and decent computational model training.

- **tu-convnext_large**

The large version has ~197M trainable parameters, [192, 384, 768, 1536] conv. channel dimensions and is built for high GPU memory capabilities and high-accuracy benchmarks and GPU setups. The large version is a much larger model and can take longer to complete training epochs due to its size. More information on this encoder can be found [here](#).

- **tu-convnextv2_base** (Default)

Further developed by Facebook AI research (2023/24), it builds upon the previous ConvNeXt encoder with several new features to improve performance; GRN (global Response Normalisation) to improve stabilise training on large-scale datasets and help to balance feature recognition, Optimized residual blocks to better constrain downsampling and attention-like reweighting to capture more global contexts, and more robust pretraining fine-tuning. The base version is comparable to its ConvNeXt predecessor in terms of trainable parameters and conv. channel dimensions and is capable of running on moderate GPU machines.

- **tu-convnextv2_large**

The larger version is also comparable to its ConvNeXt predecessor in terms of trainable parameters and conv. channel dimensions but has shown to be able to train on moderate-high and high-level GPU machines. COConvNeXtV2 has seen much better success compared to ConvNeXt and it is suggested that V2 be used over its initial version. More information on this encoder can be found [here](#).

ResNet: Residual Networks, efficient training of very deep neural networks.

- **resnet34**

Developed by Microsoft Research (2015), this deep encoder consists of an original plain-layer convolutional block network with a progressively increasing layer size; it ten integrated shortcut connections every 3x3 stage that performs identity mapping. This allows architectures to counteract the vanishing gradient problem; making training in longer neural networks more stable and thus rendering performance and degradation issues diminished significantly. The ResNet number refers to the overall depth of the encoder, usually divided into 4 stages; ResNet34 has sets of 3-4-6-3 residual blocks alongside global average pooling, each containing 2 conv. layers up to 512 layer size.

- **resnet50**

ResNet50 contains residual blocks with 3-layer conv. stages up to 1024 in the 3rd stage and 2048 in the 4th stage. Though the size is comparable to the ResNet34 network, the layer sizes allows for deeper training pathways and more global context. More information on this encoder can be found [here](#).

EfficientNet: Compound Scaling Network

- **Efficientnet-b3**

Developed by Google AI research (2020), the encoder uses a compound scaling process, which incorporated a single coefficient based on a grid search of depth, width and resolution, on top of a baseline model to generate a stable network that efficiently trains using very few parameters. It also incorporates MBConv (Mobile-inverted Bottleneck Convolutions) and SE (Squeeze-and-

Excitation) blocks to combine depth-wise and point-wise convolutions alongside a channel attention mechanisms that generates very accurate feature recognition. The block-number refers to the overall depth of the network (B0-B7), where specific variants can be chosen relative to computing capability and user case. B3 has a depth of 1 3x3, 2 3x3 and 2 5x5 conv. blocks.

- `Efficientnet-b4`

B4 has a depth of 1 3x3, 2 3x3, 2 5x5 and 3 3x3 conv. blocks.

- `Efficientnet-b5`

B5 has a depth of 1 3x3, 2 3x3, 2 5x5, 3 3x3 and 3 5x5 conv. blocks.

- `Efficientnet-b7`

B7 has a depth of 1 3x3, 2 3x3, 2 5x5, 3 3x3, 3 5x5, 4 5x5 and 1 3x3 conv. blocks. More information on this encoder can be found [here](#).

ResNeXt; Aggregated Residual Transformation Framework

- `resnext50_32x4d`

Developed my Facebook research (2017), topologically-similar stacked residual blocks are placed in a framework based on the same plain-layer encoder format of a ResNet50 model, where grouped convolutions create cardinality and mean complexity and accuracy can be maintained over larger and deeper architectures. More information on this encoder can be found [here](#).

ResNeST; Split Attention Network

- `timm-resnest50d*`

Developed my Microsoft Research (2020), it incorporates a splitting of the data into feature groups (cardinality hyperparameter) and then into separate modules funneling into a split attention block (Radix-major), focusing on featuremap attention over different groups; these then fuse and concatenate through element-wise summation across the splits and through residual shortcuts. The depth of each split is based on Resnet deep conv. network frameworks were diverse-feature datasets have been seen to perform better, more so than its ResNet counterparts. This model is based on the ResNet50 Encoder format.

- `timm-resnest101e*`

This model is based on the ResNet101 Encoder format. This encoder also requires significant compute to perform. More information on this encoder can be found [here](#).

DINO: Self-supervised Vision Encoder

- `dinov2_..`
 - `..vits14`

Developed my Meta AI Research (2020), it uses the DINO framework (Vision transformers with knowledge distillation and student-teacher network matching) alongside the addition of self-supervision, obtaining more trainable information from a smaller data input. The V2 update also uses [Sinkhorn-Knopp centering from SwAV](#) and a KoLeo regularizer leading a better performing

model without a large quantity of training data. The V2 comes in 4 sizes; vits/14 represents ViT-Small. *This encoder set is still being tested.*

- `..vitb14`

vitb/14 represents ViT-Base. *This encoder set is still being tested.*

- `..vitl14`

vitl/14 represents ViT-Large. *This encoder set is still being tested.*

- `..vitg14`

vitg/14 represents ViT-Giant. *This encoder set is still being tested.* More information on this encoder can be found [here](#).

- `dinov3_..`
 - `..vitl16`

Further developed my Meta AI Research (2025), V3 builds on the V2 network with a ConvNeXt backbone and introduces Gram Anchoring (loss term stabilisation for deep patch-wise features), High-Resolution Fine-Tuning (seperate phase for quality output) and a much larger training pool; this creates a much more powerful and versatile model but requires more compute to run. The V3 comes in many sizes, however only 2 are currently being tested; vitl/16 represents ViT-large version (the largest current non-variant model).

- `..vit7b16`

vit-7b/16 hosts an additional specialised linear classifier head. More information on this encoder can be found [here](#).

~ Encoders with an asterisk (*) are not compatible with PAN.

There are finally 9 Loss Functions that can be used when training these models: `CombinedCEDiceLoss` is the default and the most widely tested and useful function. The full loss function list includes:

- `CombinedCEDiceLoss` (Default)

Loss function that combines *Cross-Entropy* and *DiceLoss*; very robust metric accounting for class imbalance while measuring the overlap between predicted and ground truth masks.

- `BCELoss`

Binary Cross Entropy loss; metric that measures the difference between predicted probabilities and true binary labels; where labels are either 0 or 1 and lower BCE means better predictions.

- `DiceLoss`

Derived from a dice coefficient measuring the similarities between the ground truth and predicted masks; focus is on region overlap rather than pixel-specific accuracy.

- `GeneralizedDiceLoss`

Focuses on smaller inconsistencies and weights them more highly against the normal data input; rare classes are contributing more to the loss signal providing a better metric for smaller region or user case feature training.

- **CrossEntropyLoss**

Metric that calculates and predicts the probability of each class input and how close a model's predictions are to the ground truth; though this helps to inform the model of correct parts and help to suppress wrong parts. This function is great with *multiclass data*, where the loss function is minimised to guide the model to more accurate predictions.

- **TverskyLoss**

Function that deals with differentiable loss looking at pixel-wise probabilities and has been balanced to prioritise both precision and recall. This function is great with sparse-object and multiclass segmentation.

- **BoundaryDoULoss**

Boundary Difference over Union; TBC

- **BoundaryLoss**

Based on a distance metric and takes into consideration regional training information; this function is great with *highly unbalanced datasets* which have values orders of magnitude from each other.

- **ClassWeightedDiceLoss**

Handles different classes in model training by calculating the loss for each separately and then averaging it by the class frequency; this gives more credit to smaller classes and compensates for imbalance in multiclass data.